CAMPUSERP MINI-ERP

GROUP 2 — SALES & ORDER MANAGEMENT

Cross-System Integration Handoff

Prepared by Group 2: Moises, Baello, Doroja, Ruiz, Ponce, and Areolla

Revision: 23 October 2026

Audience: Owners of System 1 Catalogue, System 3 Inventory, System 4 Billing/Payment, and System 5 Delivery/Shipment.

1. Purpose and source basis

This is Group 2's handoff to the other system owners. It describes the Sales implementation, the data it supplies, the results it expects, and decisions needed for integration. It does not rewrite another group's specification or claim that another group has implemented these interfaces. System numbers identify modules; Group 2's identity is confirmed by the project team, while other teams should confirm their contacts and ownership during review.

The local Sales app supports draft-to-completion processing, GCash/bank transfer, optional COD, delivery reattempts, customer messages, manual refund/return evidence, separate issue closure, and reporting. A working local simulator is not a working cross-system integration. Keep live COD disabled until the relevant teams complete joint acceptance tests.

Source basis

The supplied System-1-and-2-Document.pdf and System12.sql are byte-identical to the packaged System_1_and_2_Document.pdf and System1_and_2.sql. Their SHA-256 hashes are respectively 29FDE37E26A486A3FD262E151892B2826D34EECDF59FA1A4953522211D26C641 and 31E7C6720A07FA2D0E14B132ACE57B544C7F3BF17DFD061D736A7445196765CD.

These sources establish the combined Catalogue/Sales baseline, not a complete Systems 1–5 schema. Additional evidence comes from the current src/ implementation, the revised Sales specification, existing integration/lifecycle notes, and packaged Inventory, Billing and Delivery references. Older notes describe prepayment-only behavior; this handoff describes current method-dependent behavior. The supplied Google Drive folder could not be accessed during preparation, so no claim is made that its current contents were reviewed.

2. Ownership: who may change what

System 1 owns categories, products, sellers, offers, prices and active status. Group 2 reads agreed Catalogue data and stores accepted purchase snapshots. Group 2 must not maintain a competing Catalogue master.

Group 2 owns sales_order, order_item and order_status_history, plus Sales-side operational facts, requests/events, payment-choice snapshots, customer preferences, conversations, notifications and resolution audit. Other systems should not directly change order_status; send authenticated results for Sales to validate. Sales-side payment/shipment facts are received evidence, not replacements for the source ledgers.

System 3 owns availability, stock reservations, expiry, release, physical deduction and restoration. System 4 owns invoices, payment verification, collection and actual refund execution. System 5 owns shipment acceptance, tracking, delivery proof and return-journey evidence. Sales coordinates these responsibilities and preserves their references.

Customer/account identity needs a shared agreement. Local Sales authentication exists, but its user/customer identifiers are not automatically valid in another system. Use agreed identifiers or explicit mapping. Never exchange passwords or browser session tokens as an integration mechanism.

3. Shared baseline SQL and identifier mapping

3.1 What System12.sql already provides

System 1 tables: category, product, seller, seller_offer. Group 2 tables: customer, sales_order, order_item, order_status_history. The v_orderable_offer read view exposes offer_id, seller_id, product_id, product_name and selling_price only when offer, product and seller are active.

Catalogue orderability is not Inventory stock availability. A row in this view must never be interpreted as a successful reservation.

The SQL uses selling_price, seller_status and surrogate offer_id. Keep seller/product uniqueness. Composite order/seller and offer/seller foreign keys enforce one seller per order. Sales preserves product_name_at_order and unit_price_at_order; line_total is generated from quantity times unit price. PHP totals use exact decimal values. Do not rename fields to current_price or remove constraints without a jointly reviewed migration.

3.2 References other teams should retain

- order_id: Sales purchase identifier; order_number is a readable reference, not a substitute foreign key.
- order_item_id: Sales line identifier. Include it in item-level reservation and shipment results.
- offer_id: Catalogue seller offer; map it explicitly to Inventory's stock record. product_id alone does not identify the seller's offer.

- customer_id and seller_id: agreed shared IDs or mapped external IDs.
- request_reference: Sales operation identity; preserve it across lookup, retry and response.
- event_reference: owning-service result identity, unique within the integration and preserved on replay.
- reservation_id, invoice_reference, transaction_reference, tracking_reference, proof_reference and fulfillment_reference: source-owned evidence references; never fabricate them in Sales.

No production identifier remapping or existing-data import was performed for this handoff.

3.3 Added Sales records

The original SQL alone does not create the entire current app. Group 2 also uses sales_user, sales_session, sales_customer_profile, sales_payment_preference, sales_order_payment, sales_order_facts, sales_outbox, sales_event, sales_simulator_result, sales_order_message, sales_delivery_update, sales_notification, sales_resolution, sales_resolution_history, sales_refund_record and sales_return_stock.

These are Group 2 application records, not proposed new operational tables for Systems 3–5. Do not give all groups unrestricted write access to them. Do not import System12.sql over an existing database as an upgrade. Review the existing application schema and additive migrations on a backed-up staging copy.

4. System 1 Catalogue handoff

Group 2 needs

A stable read contract supplying offer/product/seller IDs, product name, selling_price and orderability. The current adapter reads v_orderable_offer from MySQL; it does not currently use a general Catalogue HTTP API. If Catalogue uses another database or API, agree a replacement adapter and ID mapping rather than pretending the current view already integrates it.

Group 2 supplies or preserves

Selected offer IDs and quantities for submission validation, and immutable product-name/price snapshots for each accepted order. Catalogue price/name changes must not rewrite submitted purchases. Drafts re-check current prices and obtain customer acceptance before submission.

Joint checks

Verify active/inactive product, seller and offer combinations; duplicate seller/product offers; mixed-seller rejection; exact DECIMAL amounts; and later catalogue deactivation without silently deleting historical purchases. Catalogue must identify the authoritative source for pickup address, weights and dimensions if Delivery needs them; those fields are absent from the baseline view.

5. System 3 Inventory handoff

Sales-to-Inventory operations

reserve requests every Sales item and quantity. confirm requests confirmation of the same reservation identities. release requests release plus an authoritative guarantee that no active or delayed reservation can remain/reappear for the cancelled order.

Required responses

reserve: status reserved, future expires_at in Unix milliseconds, and reservations covering every item exactly once with order_item_id, offer_id, matching quantity and unique reservation_id. Or status rejected. Partial success is not accepted; Inventory must clean up any partial internal hold before reporting final rejection.

confirm: status confirmed, the same complete reservations, and an explicit expires_at value: null for a non-expiring confirmed hold, or a valid future Unix-millisecond deadline. Omission is invalid. Alternatively status expired. For prepayment, confirmation follows Billing verification; for COD, it follows a prepared invoice/collection arrangement without requiring advance payment.

release: status released and no_active_reservations true only after authoritative release and prevention of delayed reserve/confirm recreation. A temporary not-found observation is insufficient. Agree an order-level cancellation marker or equivalent guarantee.

Later lifecycle facts

Later expiry, release or fulfillment is a new event, not an overwritten reserve response. The implemented receiver is POST /api/events/inventory/lifecycle, authenticated with the Inventory secret. Its envelope includes schema_version 1, event_type reservation.lifecycle, event_reference, original reserve request_reference, order_id, increasing order-stream version, occurred_at in Unix milliseconds, status and exact reservation identities. Fulfilled additionally needs fulfillment_reference.

Exact duplicate lifecycle events are no-ops. Older versions are audited/ignored; conflicting versions/references are rejected. Expiry cannot invent an expired non-expiring hold. Inventory fulfillment must correspond to authorized shipment and either verified advance payment or the permitted COD arrangement. The lifecycle release event does not by itself satisfy the stronger cancellation release guarantee.

Decisions Inventory must confirm

Offer-to-stock mapping; unpaid reservation duration (the source proposes 30 minutes, not a class-approved deadline); confirmed-hold policy; COD holds; dispatch versus pickup deduction; how restoration evidence is issued; and safe cancellation of in-flight reservations. Sales never directly decrements or restores Inventory balances. See the repository lifecycle notes and actual handler for full validation; older prepayment-only sentences do not override current COD guards.

6. System 4 Billing/Payment handoff

Sales-to-Billing operations

invoice provides accepted product amounts/items, customer/order references and payment_method. verify requests an authoritative result for that invoice. cancel requests authoritative unpaid cancellation or a paid result requiring reconciliation.

Required responses

invoice: status created, nonblank invoice_reference, exact amount equal to product_total and currency PHP.

verify: status verified or failed, matching invoice_reference, exact amount and PHP currency. Verified requires a nonblank transaction_reference. Pending/unknown must stay pending and must not be reported as failed merely because a request timed out.

cancel: status cancelled_unpaid only after blocking pending/future charges and proving payment has not succeeded. If paid, return status paid with invoice_reference, transaction_reference, amount and currency. Sales preserves payment and pauses reconciliation rather than erasing the receipt.

Payment method agreement

GCash and bank transfer require verified payment before shipping. COD requires invoice preparation and valid Inventory confirmation before shipping, then verified delivery before Sales requests Billing's verify operation for cash collection. Delivery proof is not collection proof. Billing must define how a courier cash receipt becomes an authoritative transaction reference and when it can report verified.

The current definitive failed-payment path has no automatic same-order payment retry. Advance-payment failure stops fulfillment and coordinates release; a new purchase follows the appropriate cancellation/resolution process. COD failure after actual delivery retains Delivered history and becomes a payment issue, not an invented failed delivery.

Refund boundary

Sales currently records manually supplied external refund evidence with amount, method, unique reference, actor and timestamp, and rejects duplicate references/over-refunds. No Billing refund-execution API is implemented in the Sales adapter. Billing must propose and agree one before automated refunds are claimed. Issue closure is not a refund receipt and does not enter collection/refund totals by itself.

7. System 5 Delivery/Shipment handoff

Sales-to-Delivery operations

accept carries authorized order/items, seller/customer references, recipient snapshot, payment_method, collection_due for COD, and available invoice/reservation references. verify_delivery requests the actual outcome for the accepted tracking reference. A Sales request is authorization to check/process, not proof that a shipment exists or arrived.

Required responses

accept: status accepted, nonblank tracking_reference and order_item_ids exactly matching the requested lines. An acceptance failure uses status failed and the same item links.

verify_delivery: status delivered with matching tracking_reference and nonblank proof_reference, or status failed with matching tracking_reference. Delivery must verify the actual proof before returning delivered. Responses may include an appropriate customer_reason without exposing sensitive internal data.

Delay and redelivery

A delivery failure closes an attempt, not automatically the whole order. Sales can record an agreed retry and creates a distinct request suffixed :attempt-{update_id}, with redelivery true and scheduled_at as ISO UTC. Before-acceptance failure retries accept; after-acceptance failure retries verify_delivery against the existing tracking reference. If Delivery needs a new tracking reference for every retry, adapt and agree that behavior before live use; do not send a different reference to the current receiver.

The live receiver rejects a result before its scheduled attempt time. Replaying an older failed request cannot complete the new attempt. Preserve all history. Further failed attempts can be rescheduled unless fulfillment is explicitly stopped or cancelled.

Sales currently records delay/redelivery updates and customer conversations locally. No general external Delivery delay-event receiver or return-service API is promised. Agree these extensions separately. Return evidence must describe actual transit/receipt. COD nonpayment after Delivered does not prove nonreceipt or returned goods.

Missing booking inputs

The baseline does not supply weight, dimensions, pickup address, delivery service selection, courier fees or shipping-tax rules. Obtain authoritative values and agree responsibility before real booking. Group 2 currently sends product_total only; Delivery must not silently add fees to the amount Billing is asked to verify.

8. Current common adapter contract

Transport and paths

The current Sales adapter uses per-service INVENTORY_URL, BILLING_URL and DELIVERY_URL plus separate secrets. HTTPS is required outside local development. The owner services are not supplied by Group 2. Each service must support:

1. GET {service_base}/requests/{URL-encoded request_reference} to query the original operation.

2. An authenticated 404 permits POST {service_base}/requests/{action} using the original immutable JSON and Idempotency-Key equal to request_reference.

3. A signed pending result keeps the operation unresolved. A terminal result must be stable on later queries.

4. Optional signed callbacks go to the Sales host: POST /api/events/inventory, /api/events/billing or /api/events/delivery. Inventory later lifecycle events use the separate lifecycle endpoint.

These are implemented Sales adapter expectations, not evidence that another team exposes these paths. The service base URL must be supplied by its owner; no real host is invented in this document.

Authentication

Use x-sales-timestamp as UTC Unix milliseconds and x-sales-signature as lowercase hex HMAC-SHA256(secret, timestamp + '.' + exact UTF-8 body). GET signs an empty body. Timestamp freshness is within five minutes. Responses including 404 must also be authenticated. Set service clocks correctly; transmit secrets privately, not in the handoff bundle. Redirects are rejected.

Browser sessions and x-csrf-token are for local browser actions, not service callbacks. Signed live webhooks are disabled in explicit demo mode. Live mode never silently substitutes simulated success when configuration is missing.

Data and retry semantics

IDs are numeric; money is a validated two-decimal PHP string; item quantities are positive integers. Reservation deadlines/lifecycle occurrence use Unix milliseconds; redelivery scheduled_at uses ISO UTC. Result envelopes include event_reference, matching request_reference and numeric order_id plus action fields. All terminal responses must correlate to an existing Sales request.

A service request normally has one terminal outcome; later lifecycle changes need their own supported event/operation. The Sales receiver has explicit reconciliation handling for late positive payment/stock evidence. Do not treat this as permission to reuse event references with changed content. Repeated identical events are no-ops; conflicting duplicates are errors. Timeouts retain the original request; retries query before resend. Sales currently uses a 10-second HTTP timeout and a 60-second dispatch lease. Network calls occur outside database transactions.

9. End-to-end scenarios to agree and test

Advance payment

Submit validated order → Inventory reserve → Billing invoice → Billing verify → Inventory confirm → Delivery accept → Delivery verify_delivery → Sales Completed. Every arrow waits for validated source evidence. Expired/rejected stock blocks fulfillment.

COD

Submit COD order → Inventory reserve → Billing invoice/collection arrangement → Inventory confirm → Delivery accept → Delivery verify_delivery → Billing verify cash collection → Sales Completed. Before verified collection, Delivered may be true while Completed is false.

Exceptions

Unpaid cancellation requires authoritative Billing cancelled_unpaid and Inventory released/no_active_reservations acknowledgements. Paid cancellation needs an explicit Sales resolution workflow and actual external settlement evidence; it is not an automatic refund.

Delayed or failed delivery permits an eligible new attempt. Explicit cancellation/stopped unsuccessful processing blocks normal fulfillment and preserves cleanup actions. Late payment after expiry/cancellation remains real money and requires reconciliation. A Delivered COD order with failed payment stays delivered while its payment issue is resolved.

10. Resolution and reporting boundaries other teams must know

Order outcome and issue status are separate. Cancelled/Unsuccessful may each have Action required, In progress or Resolved. Only a Sales administrator can Mark issue as resolved with a note, confirmation and completed or individually documented external/not-applicable requirements. Identity, time and history persist. Closure alone never transfers money, restores stock, confirms return, changes delivery or creates a successful order.

Customer notifications and Contact Sales are existing in-app features, not an email/SMS/push integration promise. Operational teams supply reliable evidence; Sales communicates the recorded result.

Sales Overview Payments collected uses matching dated Billing verification once per order, including later-cancelled payments. Refunds recorded uses actual recorded refund receipts; Net collections is their difference. Completed-order/product reporting excludes cancelled, unsuccessful and drafts. External issue notes are not financial ledger events. Orders in final issue states must not inflate active/successful totals. Item-level returns/product refunds are unsupported, so no allocation should be invented in cross-team reports.

11. Joint acceptance checklist

For each scenario record the environment/version, teams present, request/event IDs, expected and actual status, source-ledger evidence, and pass/fail. Local unit tests and screenshots do not replace these joint tests.

- Catalogue: active offer appears; inactive seller/product/offer does not; changed price re-reviewed; historical snapshots remain.
- Inventory: full reservation succeeds; shortage rejects without leaked partial holds; mismatched items are rejected; expiry blocks shipment; confirmed non-expiring hold behaves correctly.
- Prepayment: both GCash and bank paths require exact verified amount/reference before shipment; failed payment does not ship.
- COD: disabled mode rejects COD; agreed enabled flow ships without prepayment but cannot complete before collection verification.
- Delivery: real acceptance, tracking and proof correlate to the exact items; delay remains active; two failed attempts followed by successful redelivery preserve earlier results.
- COD exception: delivered but unpaid remains Delivered with an unresolved payment issue; no automatic return is inferred.
- Cancellation: release/invoice races, late reserve, late payment and unknown timeout do not silently authorize fulfillment or erase money.
- Idempotency/security: repeated submission/request/event/refund does not duplicate work; conflicting duplicates, forged signatures, stale timestamps, customer callbacks and wrong-service secrets are rejected.
- Lifecycle: exact replay no-op; older version ignored; conflicting version rejected; restoration/deduction occurs only in Inventory and only once.
- Resolution: external refund/return/stock evidence is auditable; admin closure persists; customer cannot close; original failure/cancellation remains; cleanup is not mistaken for successful fulfillment.
- Reporting: collected/refunded totals reconcile with evidence; stopped cases stay out of success counts; dates use agreed UTC/Philippine conversion.

12. Decisions needed for integration

D01 — integration deployment: shared MySQL read view for Catalogue versus an adapter to another platform. Owners: System 1 and Group 2.

D02 — shared customer/user/seller/offer identifiers and mapping ownership. Owners: all affected teams.

D03 — reservation duration, confirmed expiry, COD stock hold, pickup/dispatch deduction and restoration. Owners: Inventory, Delivery, Group 2.

D04 — enable COD, cash receipt verification, definitive payment failure policy and refund authority. Owners: Billing, Delivery, Group 2.

D05 — redelivery scheduling/tracking policy, external delay events and return handoff. Owners: Delivery, Inventory, Group 2.

D06 — product subtotal versus freight/tax/discount and final payable amount. Owners: Catalogue, Billing, Delivery, Group 2. Current implementation remains product-only; any change requires agreed development.

D07 — source service URLs, credentials, HMAC contract, retry retention, cancellation markers, environment and role provisioning. Owners: each service owner and Group 2.

D08 — automated refund/return APIs and item-level returns. Owners: Billing, Inventory, Delivery, Group 2. Not implemented; agree as separate work.

13. Handoff contents and next meeting

Read this handoff first, then inspect schema-map.csv and examples.json. The examples use fictional identifiers and must be regenerated with real staged IDs/timestamps. System12.sql is included as a baseline reference only, not a migration. Group 2's revised specification is included for application scope. README lists source hashes, evidence limits and the precise remaining integration work.

At the next meeting each owner should bring actual schema/API samples, an agreed identifier mapping, test service URL and authentication approach, one normal and one failure response, and decisions for the items they own. Group 2 will adapt its interface after agreement; the other groups remain responsible for their own code and records.